软 件 测 试 基 础

马晓娜

2022年10月14日星期五

山东建筑大学计算机科学与技术学院软件工程专业

本次课主要内容

要点

测试用例的概念、测试用例的选择。

要点

软件测试过程模型、软件生命周期。

测试用例

- 定义
  - 针对被测试项的测试输入，执行条件和预期结果的集合
  - 完全的测试是不可能的，必须挑选出具有代表性的有效的输入组合，挑选输入组合的方法就是测试用例设计方法
  - 测试用例设计是软件测试的灵魂，体现着软件测试工程师的职业价值。测试用例可以用结构化自然语言描述，也可以用编程语言实现 。

为什么需要测试用例

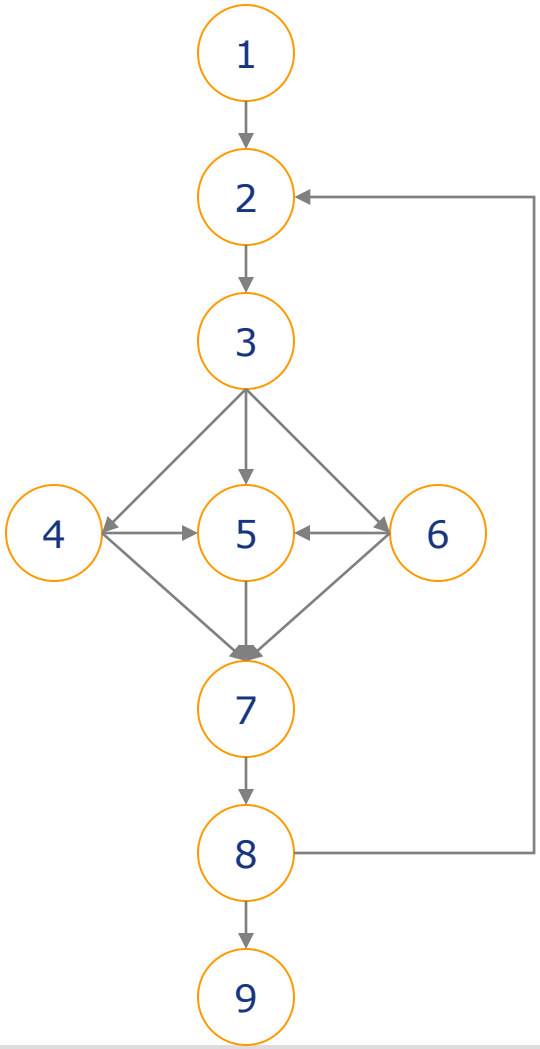
- 不可能进行完全的测试根本原因：
  - 输入的组合数目太庞大
  - 输出的可能结果数目太庞大
  - 可执行的路径数目太庞大
- 结论
  - 为了进行聪明的测试，软件测试工程师必须在众多的数据中选取恰当的做代表。

不可能进行完全的测试

- 如果你打算测试一个计算器程序的功能，你认为需要进行多少次输入？
- 结论是不计其数

1. 整型: 从 1+1 到 99999999999999999999+ 99999999999999999999
2. 小数:1.0+0.1,1.0+0.2...等等
3. 键盘上的任何一种组合
4. 为乘法和除法运算重复上面的操作

不可能进行完全的测试



- 左图是具有某程序的流程图，假设循环10次，请计算一下有多少条程序执行通路？假设由图中得到的所有路径都是可执行路径，执行一次循环大约需要10微秒（奔腾4 1.7G），且一年365天每天24小时不停机，请回答：如果循环次数为10的话，遍历图中所有路径需要多长时间？
- 结论是：$5^1+5^2+5^3+\ldots\ldots+5^{10}=6.1*10^7$，需要1分钟左右的时间
- 如果循环次数为20次，100次时，结果又如何？
- 结论是：循环20次，需要约16000小时，循环100次则大约需要$2.4*10^{48}$年

测试用例的好坏

```
#include<stdio.h>
Void main()
{
    int a[5];
    int i;
    int count=0;
    for(i=1;i<5;i++)
    {
        scanf("%d",&a[i]);
        if (a[i]==0) count++;
    }
    printf("zero: %d \n",count);
}
```

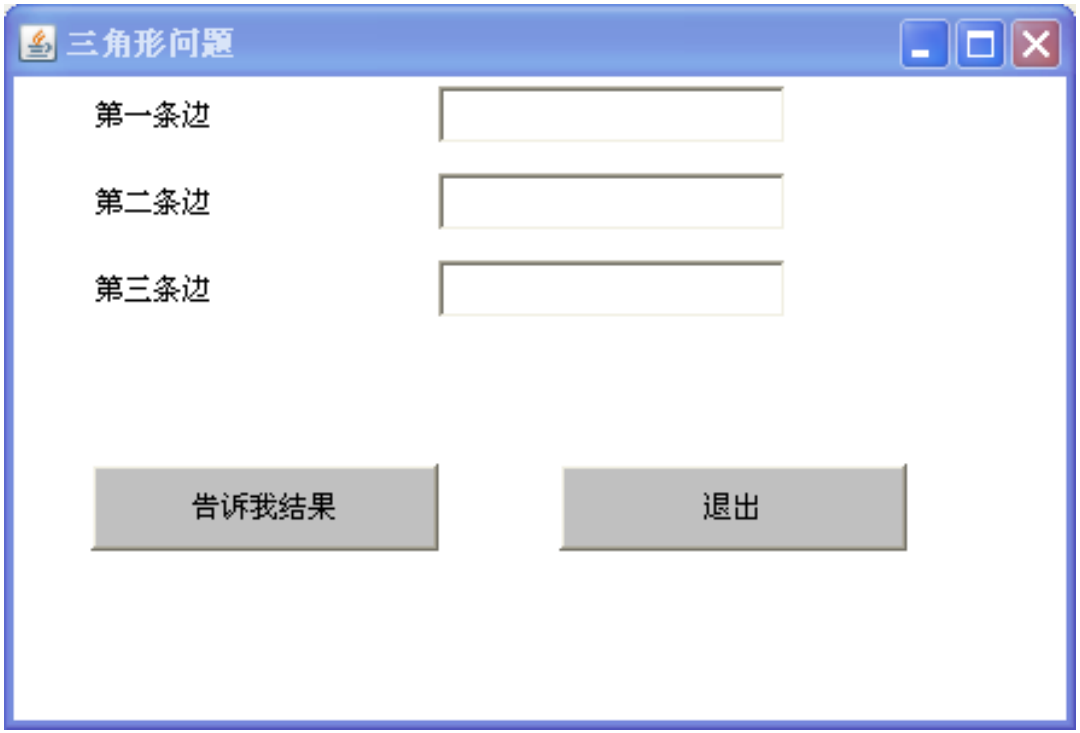
这个程序功能是计算数组中零的个数

第一次测试：输入 {1，2，3，4，0}

第二次测试：输入 {0，1，2，3，4}

三角形问题

[三角形问题]输入三个整数，这三个数分别代表三角形三条边的长度，请判断这三个数构成的三角形是等边三角形，等腰三角形还是不等边三角形并输出相应的结果。



三角形问题

判断三角形：

① 输入整数（条件）
- 输入三个整数（有效）
- 输入三个小数（无效）①

整数
- 正整数（输入三个正整数）（有效）
- 负整数（输入一个负整数，二个正整数）（无效）②
- 0（无效）③

正整数
- 二边之和 > 第三边（有效）⑤
- 二边之和 ≤ 第三边（无效）④ 1，1，2（考虑边界）

二边之和 > 第3边
- 共腰 ⑤（有效） 2，2，3
- 共边 ⑥（有效） 2，2，2
- 不共边 ⑦（有效） 3，4，5

考虑边界值
- 共腰：2，2，1 ⑧（最小）（最大）
- 共边：1，1，1 ⑨
- 不共边：~~1，2，3~~ 2，3，4 ⑩

考虑输入的最大范围 ⑪ 999999…9，99……9，9…9

② 输入非法字符

1. 空白 ⑫
2. 空格 ⑬
3. 汉字 ⑭
4. 英文字母 ⑮
6. 特殊符号 ⑯

测试用例实现

- 一个用自然语言实现的测试用例实例

用例编号	IsTriangle_ST_001
优先级	高
测试项	等边三角形判定功能
执行条件	1. 程序可以正常运行
测试过程	1. 运行程序isTriangle.class 2. 在第一条边中输入60 ,第二条边中输入60， 第三条边中输入60 3. 单击“告诉我结果I”按钮
预期结果	等边三角形
实际执行结果	

- 不同的组织可能使用不同的测试用例模版

测试用例实现

- 基于JUnit框架(www.junit.org)用Java语言实现的一个测试用例

```
import junit.framework.*;

public class TestIsTriangleTestSample extends TestCase
{
        String strTestResult;

        muckTriangleTestSample anMuck;

        public void setUp()
        {
         anMuck= new muckTriangleTestSample();
        }

        public void testbtnTell_Clicked1()
        {
          strTestResult = anMuck.muckbtnTell_Clicked(3,4,5);
          assertEquals(strTestResult,"这是一个不等边三角形!")
        }

        public void testbtnTell_Clicked2()
        {
          strTestResult = anMuck.muckbtnTell_Clicked(4,4,5);
          assertEquals(strTestResult,"这是一个等腰三角形!");
        }

        public void testbtnTell_Clicked3()
        {
         strTestResult =  anMuck.muckbtnTell_Clicked(4,4,4);
          assertEquals(strTestResult,"这是一个等边三角形!");
        }

        public void testbtnTell_Clicked4()
        {
          strTestResult = anMuck.muckbtnTell_Clicked(0,4,4); }
          assertEquals(strTestResult,"这不是一个三角形!");
        }
```

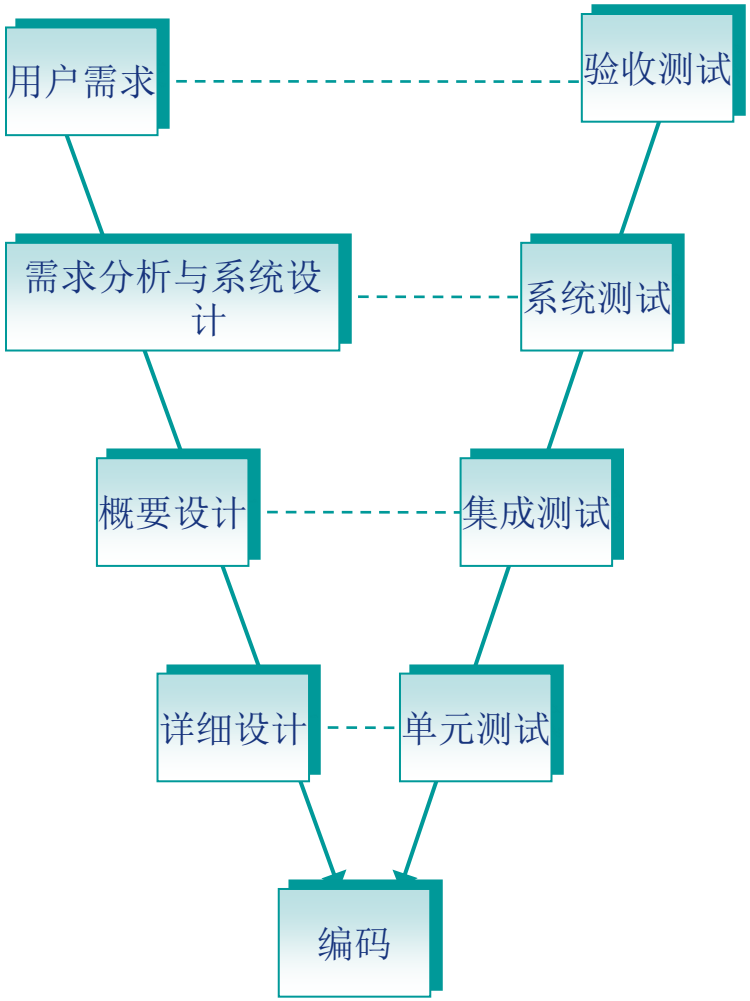
```
/*模拟单击鼠标的动作*/
private class muckTriangleTestSample
{

  boolean isTriangle = false;
  String _str;


  public String muckbtnTell_Clicked(int sideA,int sideB,int sideC
  {

     int a = sideA;
     int b = sideB;
     int c = sideC;

     if ( (a<b+c) && (b<a+c) && (c<a+b) )
        isTriangle = true;
     else
        isTriangle = false;

     if (isTriangle)
       if ((a == b) && ( b == c))
          _str = "这是一个等边三角形!";
       else if (( a != b) && ( a != c ) && ( b!=c ))
               _str = "这是一个不等边三角形!";
            else
                  _str = "这是一个等腰三角形!";
     else
          _str = "这不是一个三角形!";

     return _str;
  }
}
```

软件测试过程模型

软件测试过程模型是软件测试的工作框架，用于指导软件测试过程，在软件测试过程中使用合理的测试模型可以降低成本，提高效率。

常见的软件过程模型包括V模型和W模型。

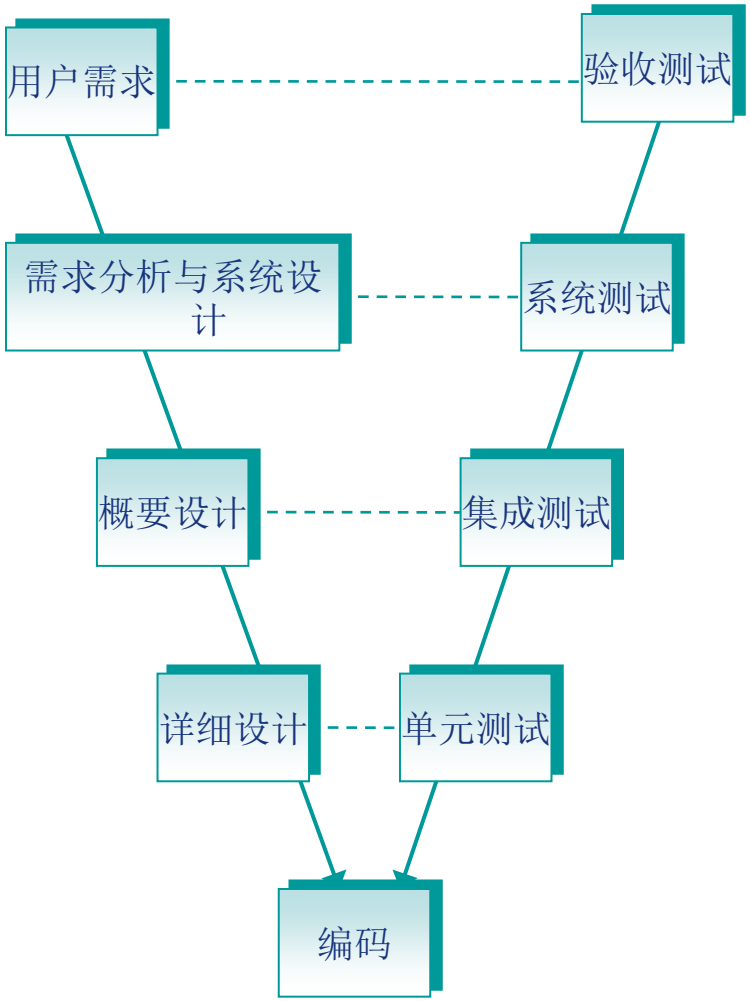
软件测试V模型



V模型最早是由Paul Rook在20世纪80年代后期提出的，在英国国家计算中心文献中发布，目的是缩短软件开发周期，提高软件开发速度。

V模型主要反映了测试活动与分析、设计的关系。

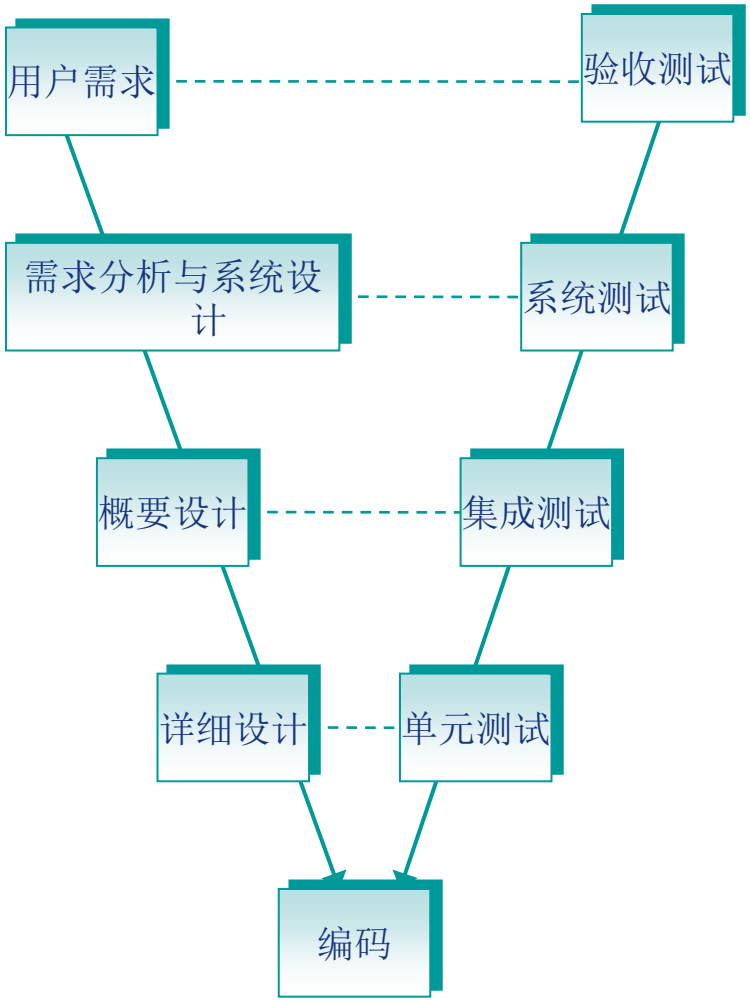
软件测试V模型



V模型清楚地描述了在那个时代的测试与开发各阶段的对应关系。

通过V模型可以看出，单元测试和集成测试主要验证程序的详细设计及概要设计，检测程序的执行是否满足软件设计要求；

软件测试V模型



系统测试是验证系统设计，检测系统功能、性能是否达到系统设计的指标；

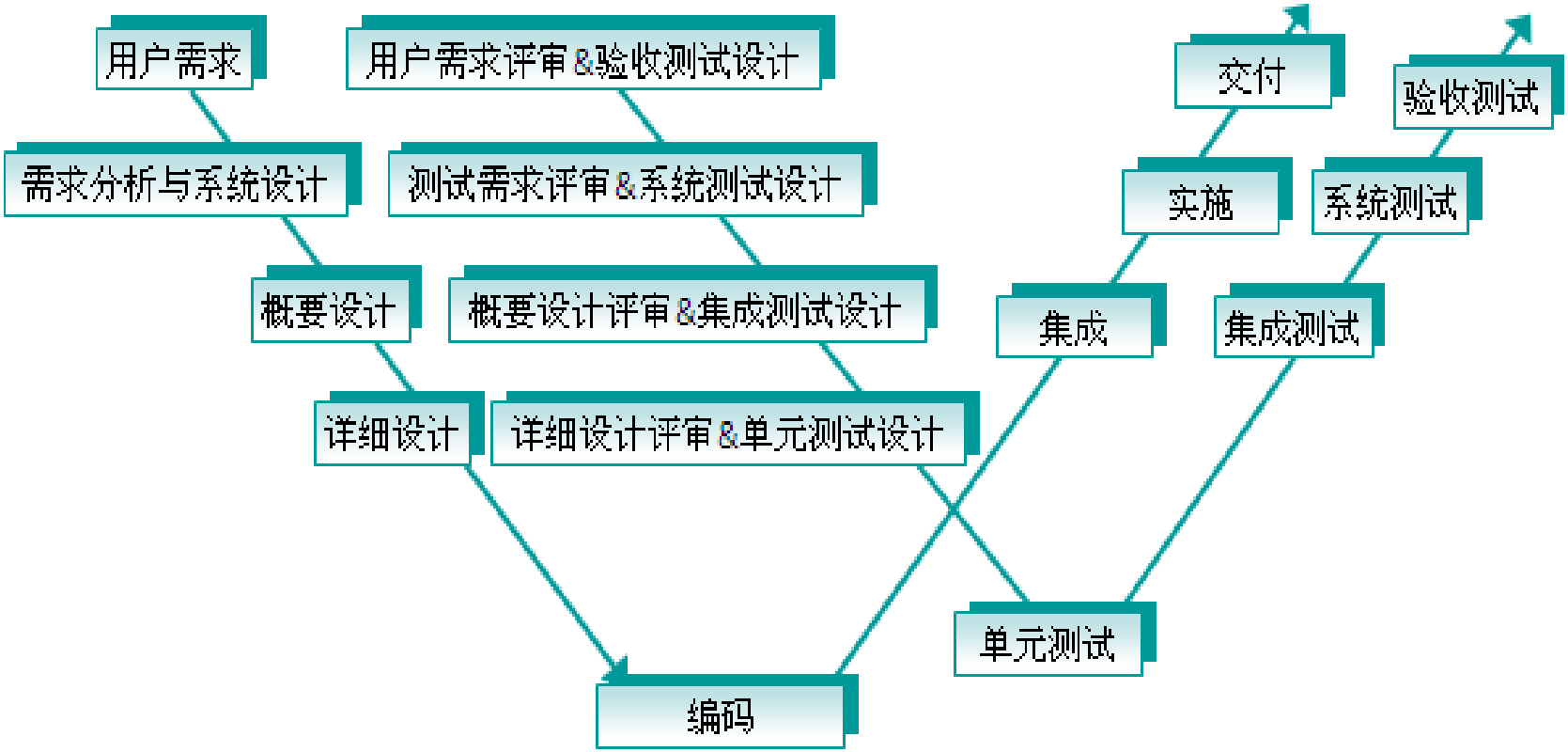
验收测试追溯软件需求说明进行测试，确定软件的实现是否满足用户需求和合同的要求。

软件测试V模型

不足：

V模型忽略了测试应该在早期介入，将软件测试过程作为在需求分析、系统设计及编码之后的一个阶段，忽略了软件测试对需求分析、程序设计验证的意义，导致需求分析及设计阶段中隐藏的问题到后期才能被发现。 所以，后面又提出了W模型。

软件测试W模型


用户需求
用户需求评审&验收测试设计
交付
验收测试
需求分析与系统设计
测试需求评审&系统测试设计
实施
系统测试
概要设计
概要设计评审&集成测试设计
集成
集成测试
详细设计
详细设计评审&单元测试设计
单元测试
编码

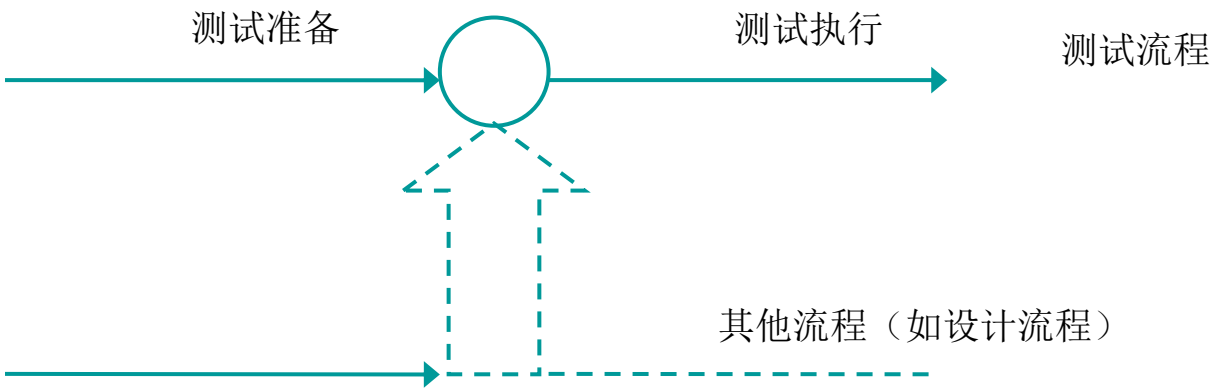
软件测试W模型

通过W模型可以看出，软件测试是伴随整个开发周期的，当相应的开发活动完成，其相应的测试便可以开始执行，且测试对象包括程序、需求、功能、设计等。相对于V模型，W模型更科学，强调了测试与开发是同步进行的，有利于尽早发现问题。

软件测试W模型

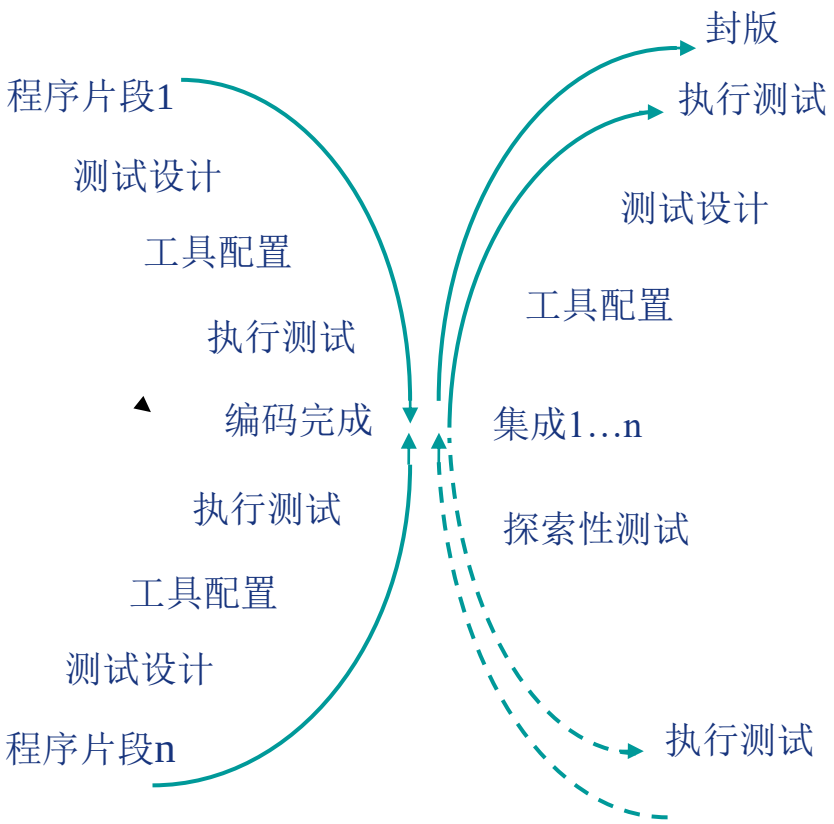
不足：W模型和V模型都把软件开发视为需求、设计、编码等一系列的串行活动，同时，开发和测试保持着一种线性的前后关系，只有上一阶段完成后，才可以开始下一阶段的活动，不能支持迭代、自发性以及变更调整。

软件测试H模型



H模型演示了在整个生产周期中某层次上的一次测试“微循环”，图中“其他流程”可以是任意开发流程，非开发流程，甚至测试流程本身。当某个测试时间点就绪时，软件测试即从测试准备阶段进入测试执行阶段。

软件测试X模型



X模型左边描述的是针对单独程序片段所进行的相互分离的编码和测试，此后将进行频繁的交接，通过集成最终合成为可执行的程序，这一点在图的右上部分得以体现。

软件测试模型总结

测试模型	特点
V模型	V模型强调软件开发过程中需要经历若干个测试级别，并将每个测试级别与开发级别相对应。但该模型把软件测试作为编码完成之后的活动，忽略了需求分析与设计等前期阶段的测试，导致部分错误直到后期才能发现。
W模型	W模型补充了V模型的不足，强调了软件开发过程与测试过程并行进行。但该模型仍把软件开发视为串行活动，同时，开发和测试保持着线性的前后关系，不能支持迭代、自发性以及变更调整。

软件测试模型总结

X模型	X模型是对V模型的改进，该模型提出针对单独的程序片段进行相互分离的编码和测试，此后通过频繁的交接，通过集成最终合成为可执行的程序。X模型并不要求对每一个程序片段都进行单元测试，但模型没有提供是否要跳过单元测试的判断准则。
H模型	H模型强调测试是独立的，只要测试准备完成，就可以执行测试，体现了测试流程的完整性。

软件测试生命周期

编号	阶段	活动	输出产物
5.1	测试计划	创建一个总体的测试计划，对软件测试活动作出整体规划。	测试计划、风险评估表、测试策略
5.2	测试分析	创建一个更加细致的测试计划，提炼质量需求形成测试需求（功能性）矩阵。	重新修订完善的测试计划、测试需求矩阵
5.3	测试设计	设计完善测试用例，决定哪些用例自动化执行，哪些手工执行。	测试用例、 测试数据
		自动化测试脚本编制。	测试规程/测试脚本、驱动器

软件测试生命周期

5.4	测试执行与回归测试	按照测试计划要求执行测试并进行回归测试	测试结果、缺陷报告、测试执行概要、测试过程改进计划（选项）
5.5	测试评估	评估测试过程，编写测试报告	测试报告，包括：测试日志、测试记录、缺陷报告和测试总结报告。

作业题

1.什么是V模型和W模型，模型的特点分别是什么？

答案：

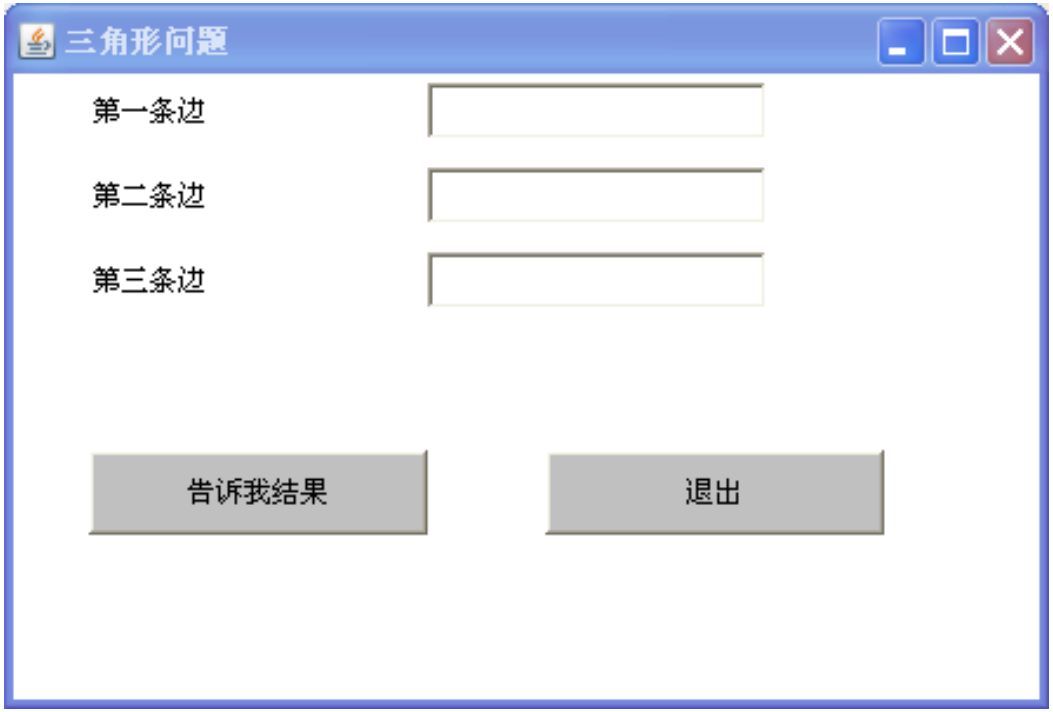
2.给出软件测试生命周期过程？

答案：

作业题

三角形问题的测试用例设计：

[三角形问题]输入三个整数，这三个数分别代表三角形三条边的长度，请判断这三个数构成的三角形是等边三角形，等腰三角形还是不等边三角形并输出相应的结果。



Thank You !